

WaterOS 初赛汇报

Rust - RISC-V64 / LoongArch64 双架构内核

参赛队名：待填写
队伍成员：待填写
指导老师：待填写

山东大学（青岛）

汇报内容

- WaterOS 介绍：项目实现概览
- 总体架构：分层设计与双架构适配
- 模块设计：启动、内存、任务、文件系统、系统调用、IPC 与网络
- 总结与展望：当前特色、后续完善方向

01 WaterOS 介绍

项目实施概览

项目实现

WaterOS 是基于 Rust no_std 环境实现的操作系统内核，支持 RISC-V64 与 LoongArch64 双架构启动和用户态程序运行。

RISC-V64

LoongArch64

Rust no_std

Linux syscall ABI

VFS / ext4 / procfs

- 建立从平台入口到用户程序运行的完整主路径
- 在两种指令集架构上保持统一的用户态对象模型
- 支持进程、地址空间、文件描述符、IPC、网络 and 日志等核心机制
- 使用组件化 crate 组织内核，降低模块之间的耦合

02 总体架构

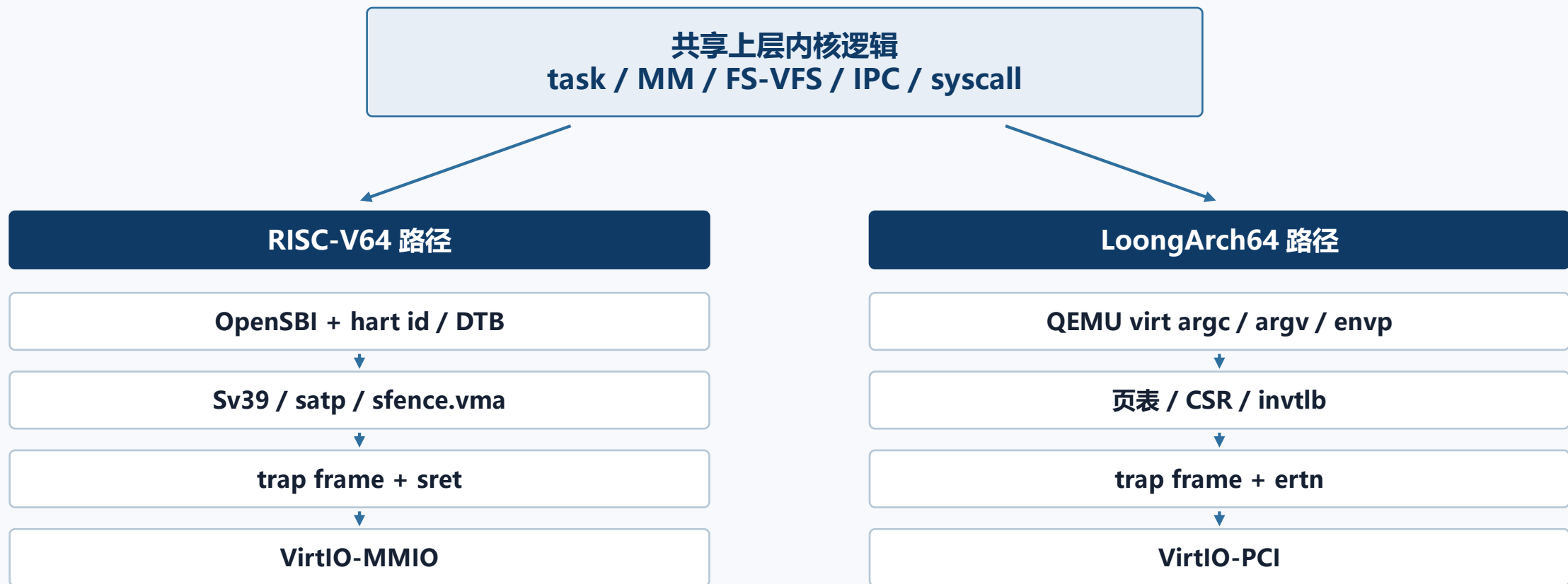
分层设计与双架构适配

分层架构

用户层	用户程序 / busybox / libc / 脚本入口	
ABI 层	Linux generic 64-bit syscall ABI	
内核核心	syscall task mm vfs fs ipc net cred klog	
驱动层	driver block char net virtio	
平台层	platform arch trap timer firmware	
机器层	RISC-V64 QEMU virt	LoongArch64 QEMU virt

上层模块围绕用户态可见语义组织，底层模块负责吸收架构、固件、中断、页表和设备总线差异。

双架构适配



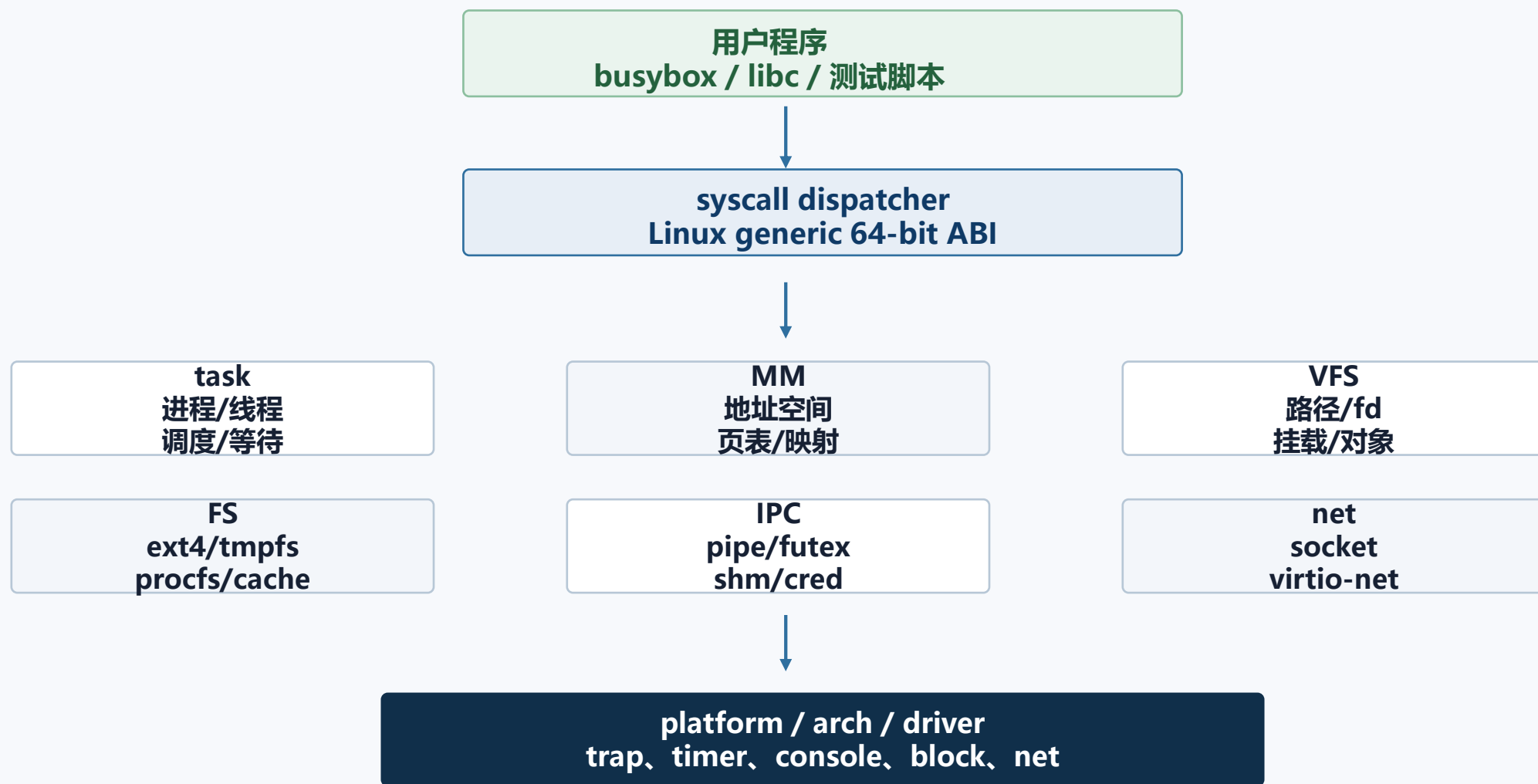
两条路径保留各自的启动、页表/TLB、trap frame 和设备总线实现；进入上层内核后使用同一套对象模型

。

03 模块设计

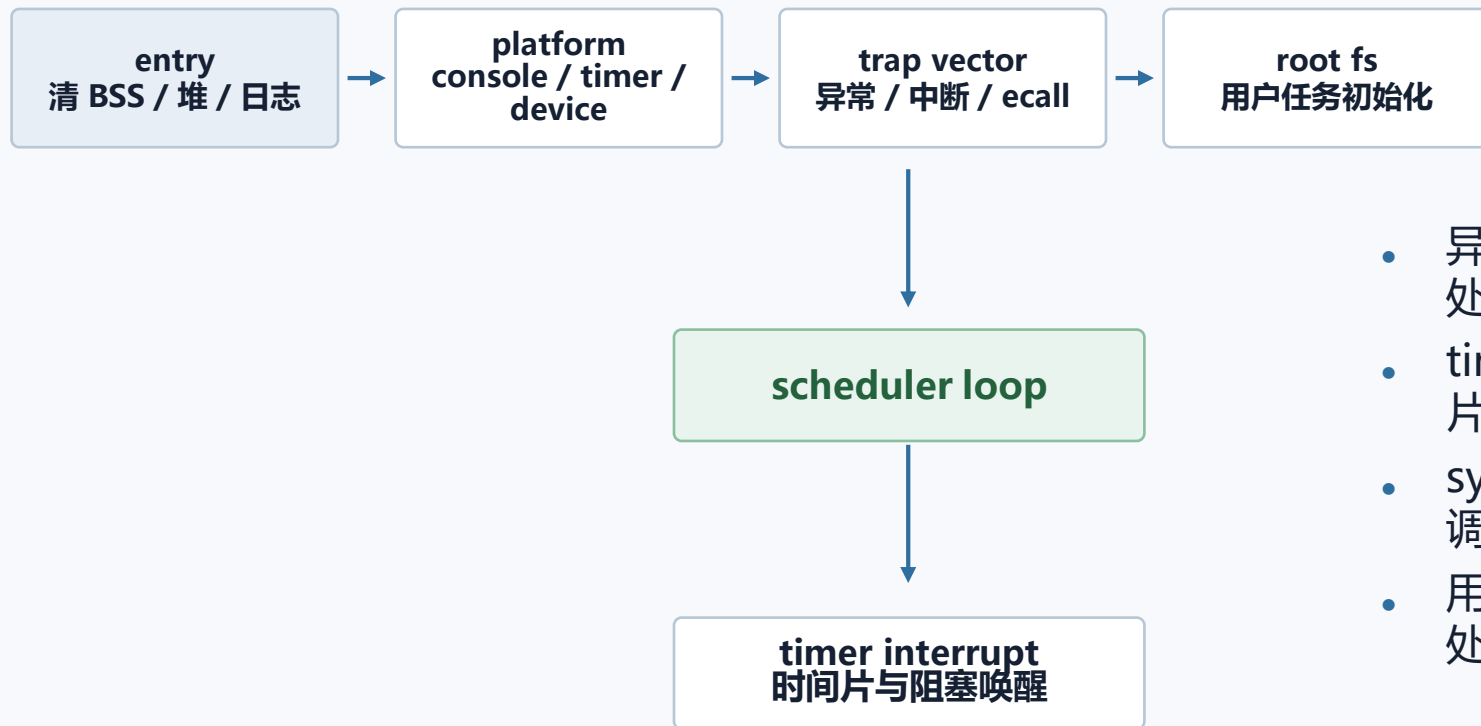
启动、MM、task、FS/VFS、syscall、IPC 与网络

模块设计总览



WaterOS 的模块拆分围绕用户程序运行路径展开：系统调用进入内核后，按照资源类型分派到 task、MM、VFS、FS、IPC 和网络等子系统；平台与驱动层负责处理底层差异。

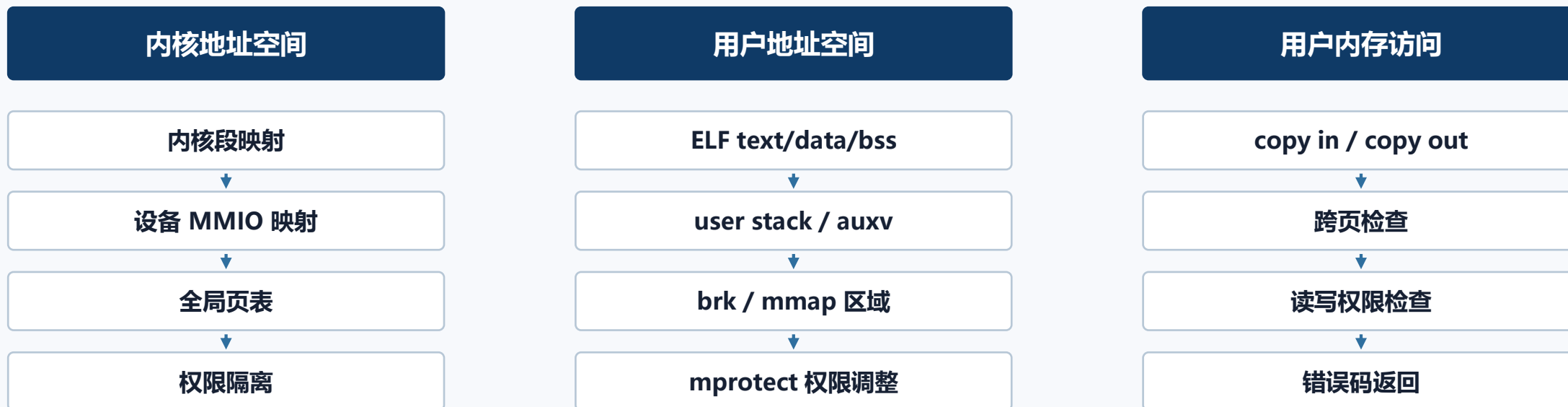
启动、异常与系统调用入口



- 异常、中断和系统调用统一进入 trap 处理路径
- timer interrupt 推动调度器获得时间片
- syscall 根据 ABI 编号进入分发表，再调用 task、MM、VFS、IPC 等子系统
- 用户态返回值统一按 Linux errno 语义处理

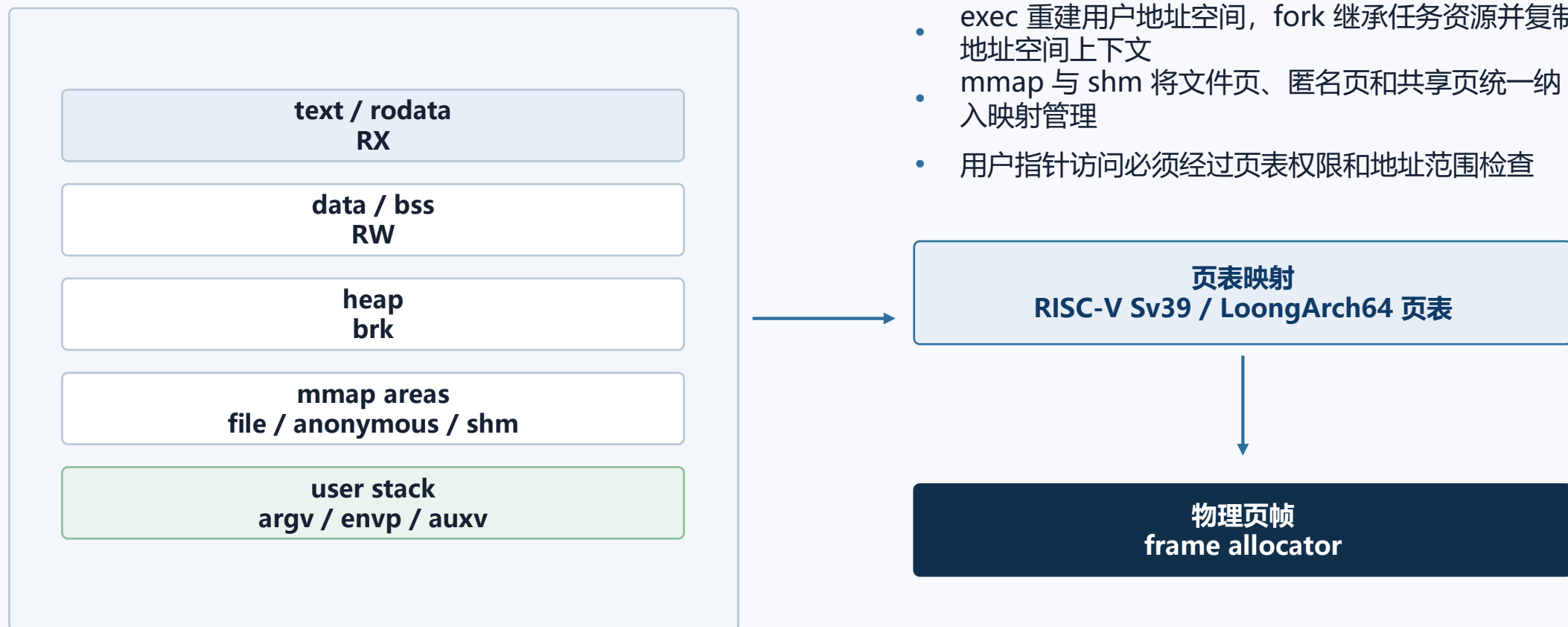
启动阶段完成基础运行环境和设备路径初始化；进入调度循环后，用户态 ecall、外部中断和 timer interrupt 都由 trap 路径统一接入。

内存管理



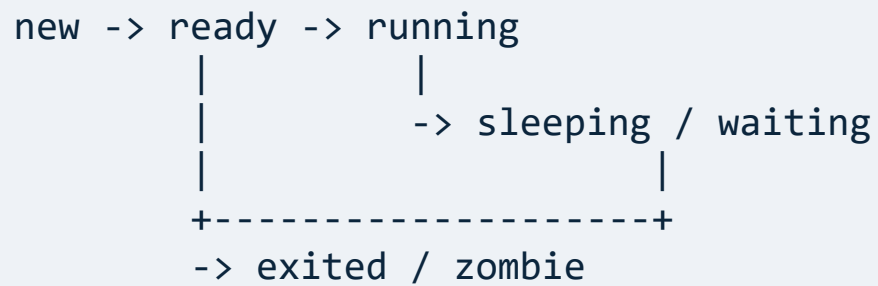
内存子系统把页表机制、ELF 装载、VMA 调整和用户指针访问放在同一条权限边界内处理。

地址空间与映射对象



地址空间对象保存用户程序可见的虚拟内存布局，页表对象负责把这些区域落实到物理页帧；系统调用只通过受控接口访问用户缓冲区。

任务管理与调度



- task 保存 trap frame、地址空间、文件描述符表、cwd、cred 和 IPC 资源
- fork 复制任务上下文和可继承资源，exec 替换地址空间并重建用户栈
- wait 回收子进程退出状态，避免资源长期挂在任务树上
- 调度器维护运行队列，timer interrupt 与阻塞唤醒共同推动状态切换

任务对象维护的核心资源



进程创建与资源继承



进程管理的重点是保持任务上下文、地址空间和可继承资源的一致性。fork 负责复制执行现场，exec 负责替换用户镜像，wait 负责回收退出状态。

资源	fork	exec	exit/wait
地址空间	复制或重建上下文	装载新 ELF	释放用户映射
fd 表	按继承规则复制	处理 CLOEXEC	关闭并释放引用
cwd / root	继承路径上下文	继续沿用	随任务释放
cred	继承身份信息	保留权限上下文	随任务释放
子进程关系	挂入任务树	保持父子关系	wait 回收退出码

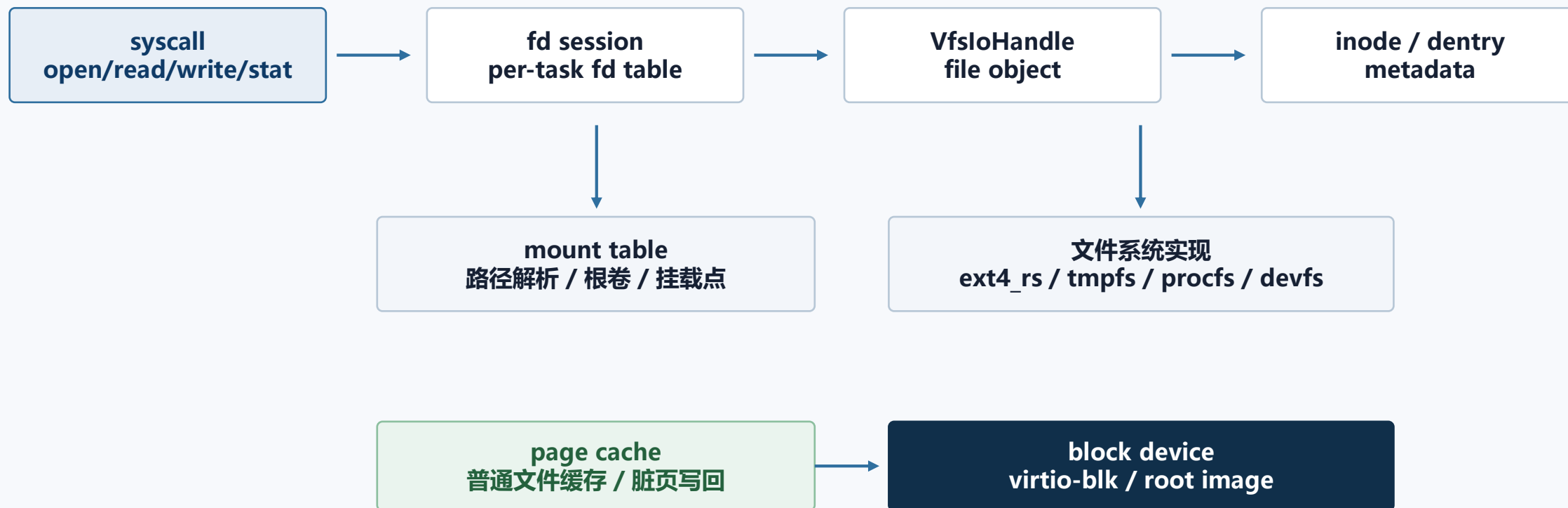
文件系统与 VFS

层次	职责
wateros-fs	ext4 根卷、tmpfs、procfs、块设备挂载
wateros-vfs	路径解析、mount table、dentry/inode、fd session
page cache	普通文件缓存、脏页管理、减少块设备重复访问
syscall	open/read/write/stat/mount/dup/pipe 等用户态接口

procfs 提供 /proc/<pid>/stat、status、cmdline、/proc/meminfo、/proc/mounts 等只读视图。

VFS 让系统调用层只依赖统一文件对象，具体文件系统、设备文件和 procfs 节点通过同一套 fd 读写接口向用户态暴露。

VFS 对象关系



fd 层只关心统一文件对象；具体读写、目录、procfs 视图和块设备访问由下层对象承接。

系统调用兼容层

ecall

```
-> syscall id  
-> fetch args  
-> validate user pointer / fd / flags  
-> call kernel subsystem  
-> return value or -errno
```

- 采用 Linux generic 64-bit syscall 编号和参数约定
- 对文件、进程、内存、时间、futex、socket 等路径分别实现语义转换
- stat/fstat/statx 从 VFS metadata 填充结构体，保证用户态看到真实对象信息
- 未支持的 flag、非法 fd、越界用户指针和权限错误返回明确 errno

系统调用层承担 ABI 边界转换：从寄存器取参数，检查用户地址、fd 和 flag，再把请求转交给内核对象，最后按 Linux 语义返回数值或负错误码。

设备与驱动路径

syscall	read / write / ioctl / socket / mount	
对象层	console fd block device socket fd procfs view	
驱动抽象	char device block device net device irq handler	
VirtIO	virtio-blk virtio-net queue descriptor interrupt	
平台总线	RISC-V VirtIO-MMIO	LoongArch64 VirtIO-PCI

- 字符设备负责 console 和日志读写，块设备承接根文件系统和 page cache
- 网络设备接入 smoltcp, socket 再通过 fd 表暴露给用户态
- 平台层处理设备发现、中断和总线差异，上层驱动接口保持一致

IPC、网络与诊断能力

IPC 与同步

- pipe 统一接入 fd 读写路径
- futex 支持用户态等待与唤醒
- shm 维护共享内存对象和映射生命周期
- cred 参与 uid/gid、capability 和权限检查

网络与诊断

- virtio-net 接入 smoltcp 协议栈
- socket 通过 fd 表向用户态暴露
- klog 和 syslog 提供内核日志读取路径
- QEMU 日志、PC 监视和符号解析辅助定位问题

IPC、网络 and 诊断能力都尽量接入已有对象模型：pipe、socket 和 klog 可以作为 fd 操作，futex 与 shm 则围绕等待队列和地址空间映射维护生命周期。

04 总结与展望

项目特色与后续完善方向

项目特色

- 双架构主路径：RISC-V64 与 LoongArch64 都具备启动、页表、设备、文件系统和用户态运行路径
- 组件边界清晰：平台、驱动、MM、task、FS/VFS、syscall 通过 API 层连接
- 用户态路径完整：ELF 装载、地址空间切换、syscall、fd、pipe、procfs、socket 已形成连续链路
- Linux 兼容方向明确：系统调用编号、参数、返回值、errno 和文件对象语义按 Linux generic 64-bit ABI 收敛
- 可诊断性较强：自检、日志、QEMU 输出和脚本入口贯穿启动到用户程序运行

后续展望

- 继续补齐两种架构下的长时间运行一致性验证
- 完善线程、信号、futex、调度策略和资源回收语义
- 强化 ext4 写路径、page cache、mount 和目录树一致性
- 完善网络 socket 生命周期、错误处理和并发场景
- 继续整理文档说明

谢谢

WaterOS